Les Algorithmes Informatiques

Christelle Caillouet, Dino Lopez Pacheco – {christelle.caillouet, dino.lopez}@univ-cotedazur.fr

1 Problèmes, Algorithmes et Programmes

Un **problème** est une tâche qui doit être effectué, souvent en prenant en compte plusieurs contraintes. Un problème est décrit par son état initial et la sortie recherché.

Un **algorithme** est une méthode générale pour résoudre un ensemble de problèmes. Un algorithme doit être correcte, composé d'un nombre fini des étapes précises vers la solution du problème, sans ambiguïté, et son exécution doit finir dans un instant donné. Un algorithme est dit correct lorsque, pour chaque instance du problème, il se termine en produisant la bonne sortie, c'est-à-dire qu'il résout le problème posé. On mesure l'efficacité d'un algorithme notamment par sa durée de calcul, par la précision des résultats etc.

Un algorithme sera performant s'il utilise avec parcimonie les ressources dont il dispose. L'analyse de la complexité algorithmique permet de prédire l'évolution en temps calcul nécessaire pour amener un algorithme à son terme, en fonction de la quantité de données à traiter.

Un **programme** est la réalisation concrète d'un algorithme.

2 Construire un algorithme

2.1 Exemple : le four

Pour construire un algorithme, il faut tout d'abord passer par une étape d'abstraction qui permet de prendre en compte les contraintes du système et outils dont on peut tirer parti. Considérons par exemple le problème suivant : comment maintenir un four à la température souhaitée par un utilisateur. Voici à quoi il pourrait ressembler l'énoncé de la solution (au plus simple possible) :

> *Voir quelle température a été choisi par l'utilisateur. Ensuite, si la température du four est bas par rapport à la température souhaitée, activer fortement la résistance chauffante du four. Si la température est égale à la température souhaitée, activer très faiblement la résistance chauffante du four afin d'empêcher son refroidissement. Enfin, si la température du four est supérieure à la température souhaitée, désactiver la résistance chauffante. Après avoir pris une décision sur l'activation ou désactivation du four, atteindre 60 seconds, puis répéter la même opération tant que le four est allumé.*

Identifions ensuite les informations importantes qui rendront notre solution générale : identifions les variables du système. Ici les variables sont la température demandée par l'utilisateur du four (que l'on peut noter « `Tu` ») et la température du four (que l'on peut noter « `Tf` »). Dans le schéma, on fait aussi intervenir un laps de temps, auquel on peut associer une variable « `t` ». On peut alors réécrire une suite d'instructions un peu plus codifiées. Notre solution devient donc

> *Lire la température Tu choisi par l'utilisateur. Ensuite, si la température du four Tf est bas par rapport à Tu, activer fortement la résistance chauffante du four. Si Tf est égal à Tu, activer très faiblement la résistance chauffante du four afin d'empêcher son refroidissement. Enfin, si Tf est plus grand que Tu, désactiver la*

résistance chauffante. Après avoir pris une décision sur l'activation ou désactivation du four, atteindre t seconds (60s), puis répéter la même opération tant que le four est allumé.

Enfin, pour traduire cela en langage informatique, on simplifie au maximum les phrases en utilisant un vocabulaire très basique qui correspond à des opérations élémentaires similaire au langage connues par la machine. Nous aurons enfin un algorithme sous la forme de pseudocode, dernière étape avant la programmation de la solution.

Voici donc l'algorithme recherché, en pseudocode. Les mots en verts sont des fonctions : des instructions qui lancent un autre algorithme, défini ailleurs. Notez qu'on ajoute des parenthèses à la fin de ces fonctions pour les différencier d'une instruction simple. Ces parenthèses permettent d'une part de savoir que cette instruction fait appel à une fonctions (plusieurs autres instructions), et d'autre part, à indiquer si des variables ou valeurs doivent être envoyé à ces fonctions. Dans l'algorithme ci-dessous, les fonctions gérant le chauffage du four ne reçoivent pas d'information pour leur exécution. La fonction attendre() reçoit cependant une valeur qui lui permet de savoir combien de temps cette fonction doit attendre avant de « réveiller » à nouveau la fonction chauffage du four.

```
Fonction lancement_four(){
  // variables du système
  Tu : température du four souhaité par l'utilisateur (entier)
  Tf : température courante du four (entier)
  t = 60 : temps en sec avant de vérifier l'état actuel du four (entier)

  Lire Tu
  tant que (four_est_allumé()) {
    Lire Tf
    Si (Tf < Tu){
      activer_resistance_fortement()
    } Sinon {
      Si (Tf == Tu){
        activer_resistance_faiblement()
      } Sinon {
        desactiver_resistance()
      }
    }
    attendre(t)
  }
}
```

2.2 Pourquoi écrire des algorithmes

Écrire un algorithme est une étape très importante. **D'une part, écrire un algorithme permet d'appréhender les contraintes de la solution, optimiser notre solution afin d'utiliser un nombre minimal d'instructions à exécuter sans se soucier des erreurs propres à la programmation. Nous gagnons donc en efficacité.** De plus, quand on développe un système grand, composer de milliers de lignes de code, **l'algorithme permet à l'équipe de développement (composé par fois par ingénieurs sur plusieurs pays et utilisant des langages des programmation très différents) de travailler de manière parallèle, en analysant les parties du système indépendantes l'une des autres.** Encore une fois, notre algorithme nous permet de gagner en efficacité.

3 Écritures des pseudocodes

Il n'y a pas de règles strictes pour écrire un pseudocode (i.e. l'algorithme). Nous pouvons cependant adopter quelques règles de facto souvent utilisées.

3.1 Écrire des fonctions, commentaires et interaction avec l'utilisateur

Une fonction est souvent déclarée avec le mot clé « fonction » ou « function » selon la langue choisie par le développeur, suivi du nom de la fonction et des parenthèses où en indique les arguments d'entrée. Les traitements à exécuter par la fonction est entourés par des accolades.

Si la fonction doit renvoyer des informations après leur exécution, on utilise le mot clé « retour ». Exemples :

```
fonction somme(a,b) {
   // si malgré tout, je pense que mon pseudocode n'est pas assez claire,
   // je peux introduire des commentaires à mon algorithme, comme ici
   // la variable c garde la somme
    c = a + b ; // commentaire dans la même ligne d'une instruction!!
   // et on renvoie le résultat
   retour c
}
```

Quelque part, on aurait d'autres morceaux pseudocodes avec une ligne de type « res = somme(3, 4) ».

```
fonction demande_age(nom) {
    affiche « Bienvenu. Aujourd'hui c'est le » + date()
    affiche « quel est votre âge » + nom + « ? »
    lire age ; // « age » -> variable contenant âge rentrée par l'utilisateur
}
```

3.2 Bloques conditionnels

Pour écrire une condition, on peut utiliser

```
si (conditions) {
    …
} sinon { ; // sinon est optionnel
    …
}
```

Ou

```
si (conditions) {
    …
} sinon si (conditions) {
    …
} ; // sinon ou rien, au besoin
```

On peut vérifier quelle valeur prend une variable avec un « si » de type « switch » (nous verrons plus tard ce que c'est un switch)

```
Si (var) égal à
    « a ») { … faire quelque chose}
    « b ») { … faire autre chose}
    Autre chose {… comportement par défaut}
Fin si
```

3.3 Les boucles

On peut écrire plusieurs types des boucles, pour l'exécution de tâches répétitives et selon la solution à proposer.

```
Faire tant que (conditions) {
    … ; // chose utile pour ma fonction
}
```

Ou sa variante

```
Faire {
…
} tant que (conditions)
```

Boucle « pour », permettant de déclarer une boucle avec des instruction d'initialisation et des instructions à exécuter avant de rentrer à nouveau dans la boucle.

```
Pour (variable) allant de (début) à (fin) {
 }
```

Exemple :

```
pour i allant de 0 à n {
  …
  i = i + 2 ; // on augmente la valeur de i de 2 unités
  // si i augmentait d'une unité, on aurait pu omettre la ligne
  // i = i + 1
}
```

3.4 Tableaux

Un tableau en pseudocode est représenté avec la notation « nomTableau[i] », ce qui indique qu'on accède au contenu de la position « i » du tableau « nomTableau ». Le premier élément d'un tableau se trouve à l'index zéro et le dernier élément, à l'index « n-1 ».

Le tableau peut avoir une seule ou plusieurs dimensions. Voici comment déclare-t-on fréquemment un tableau dans un algorithme.

```
tab = {} ;// tableau vide
tab = {"a","b","c"} ; // tab[0] sera égal à « a »
tab = {{1,"a"},{2,"b"},{3,"c"}} ; // tab[2,0] sera égal à « 3 »
tab1 = tab[2] ;// équivalent à tab1 = {3,"c"}
tab[2,0]=4 ; // on a changé « 3 » par « 4 ».
```

4 Complexité d'un algorithme

Nous avons écrit qu'un algorithme doit être performant et efficace. Il y a plusieurs facteurs qui peuvent être pris en compte pour évaluer ces aspects. Souvent, les 2 paramètres principales sont le temps d'exécution et la mémoire. Dans l'ère du *big data*, un programme gaspillant ce type de ressources peut être juste très mauvais.

Le temps d'exécution dépend des caractéristiques de la machine bien sûr. Cependant, il y a de programmes (et donc des algorithmes) qui seront lents à l'exécution indépendamment du matériel utilisé. Pour avoir une idée du temps d'exécution d'un algorithme, nous pouvons calculer sa « complexité ». Notez donc que la complexité d'un algorithme n'indique pas la difficulté que l'auteur aurait eu pour la concevoir, mais le temps attendu pour son exécution.

La complexité on la note souvent O(f(x)) (Big-O) et décrit le comportement de la fonction f(x) quand x tend vers l'infini. Nous allons voir maintenant comment arriver à estimer O(f(x)) pour nos algorithmes.

Voici l'algorithme pour chanter « Alouette, gentille alouette ».

```
1. Fonction chante_allouette(partie[0..n-1]){
2.   Pour i allant de 0 à (n-1){
3.     Chante « Alouette, gentille alouette »
```

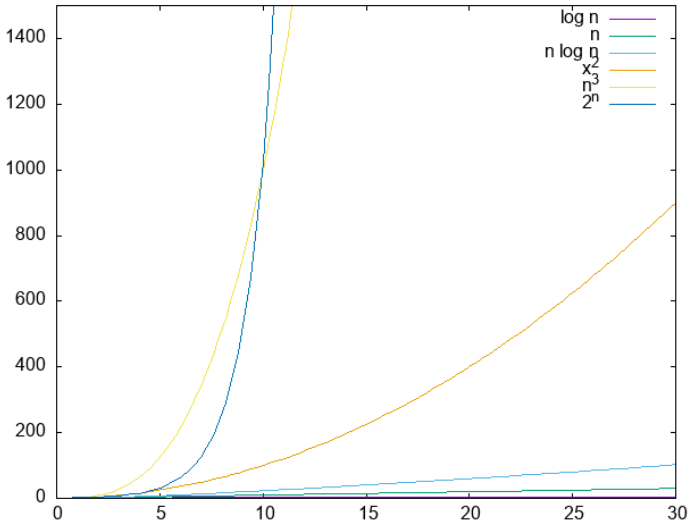
```
4.      Chante « Alouette, je te plumerai »
5.      Pour j allant de 0 à 1 {
6.        Chante « Je te plumerai » + partie[i]
7.      }
8.      Pour j allant de i à 0 {
9.        Chante « Et » + partie[j] + « ! Et » + partie[j] + « ! »
10.     }
11.     Chante « Alouette, Alouette! »
12.     Chante « Oh, oh, oh, oh »
13.   }
14.}
```

Pour estimer la complexité d' « Alouette, gentille alouette », on serait tenté de compter le temps d'exécution de chaque instruction. Cependant, supposant un nombre « n » très grand d'éléments du tableau, le temps d'exécution de l'algorithme sera principalement affecté par cette valeur « n ». En conséquence, le temps d'instruction ajouté par les constantes (instructions « chante », lecture/écriture de variables) est négligé pour le calcul de O(f(x)), et la complexité de notre algorithme devons la calculer sur les blocs d'instructions en fonction de « n », donc, les lignes 2 et 8. Il faut donc estimer la complexité à partir de l'équation « « $\sum_{i=1}^{n} i$ ». Ceci donne donc « $n(n+1)/2$ ».

Suivant notre approche de simplification (négligeant les constantes), on récrira notre équation comme étant seulement « n^2 ». Donc, la complexité de notre algorithme « Alouette, gentille alouette » est $O(n^2)$.

Quand on estime la complexité d'un algorithme, on simplifie jusqu'à obtenir l'une des fonctions suivantes, en prenant uniquement le facteur le plus fort :

- O(1) – constante
- O(log n) – logarithmique
- O(n) – linéaire
- O(n log n) – n log n
- O(n^2) – quadratique
- O(n^k) – polynomiale (k >= 1)
- O(a^n) – exponentielle



En générale, un algorithme performant ne doit pas avoir une complexité exponentielle (même si tout dépend des contraintes de la vie réelle).

5 Exercice

1. Écrire un algorithme effectuant le décalage des éléments d'un tableau contenant « n » éléments. Donnez la complexité de votre algorithme. Exemple : Tableau initial :

D	E	C	A	L	A	G	E

Tableau modifié (décalage à gauche) :

E	C	A	L	A	G	E	D

2. Supposez qu'on a 2 entiers représentés sous la forme des tableaux X[0..m-1] et Y[0..n-1], représentant les chiffres $x = \sum_{i=0}^{m-1} X[i] \cdot 10^i$ et $y = \sum_{j=0}^{n-1} Y[j] \cdot 10^j$. Le produit xy peut être calculé avec la formule $y.x = \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} (X[i] \cdot Y[j] \cdot 10^{i+j})$. Écrivez le pseudocode qui calcule et renvoie le produit xy, 2 tableaux reçus en argument de la fonction. Utilisez uniquement les opération arithmétiques +,-,x,/,% de base uniquement. Quelle complexité aura votre algorithme ?

3. Tout nombre entier n>1, admet une décomposition unique en facteurs premiers. Par exemple, 60 = 2x2x3x5. Un algorithme simple pour trouver cette décomposition peut être décrit de la façon suivante : Soit (p1, p2, ... pk) la suite croissante des facteurs premiers de n. Supposons que l'on connaisse les j premiers facteurs premiers de n et le nombre m tel que n=p1×p2×...×pj×m. Pour trouver le (j+1)ième facteur premier de n, on teste successivement pour tous les entiers d, tel que pj≤d≤m, si d divise m. Le premier entier trouvé est p(j+1).

   Écrivez l'algorithme d'une fonction qui renvoie un tableau contenant la décomposition en facteurs premiers d'un entier « n » reçu en argument. Si « n » est premier, elle n'affiche que « n ». Indication : La suite croissante (p1, p2, ... pk) des facteurs premiers de n n'est pas strictement croissante car on peut avoir pi=pi+1 pour certains i, 1≤i<k (par exemple, 60=2×2×3×5).

   Estimé la complexité de votre algorithme.

4. Soient x un réel et n un entier strictement positif. On peut calculer une approximation de sin(x) en utilisant la formule suivante : sin(x) = x - x3/3! + x5/5! - x7/7! + ... On dit que sin(x) est calculé à l'ordre n si la somme correspondante contient n termes. En vous limitant aux quatre opérateurs arithmétiques +, -, × et /, donnez un algorithme qui, pour tout réel x et pour tout entier n strictement positif, retourne la valeur sin(x) à l'ordre n. Indication : Pour calculer efficacement sin(x) à un ordre n élevé, il ne faut pas recalculer les différentes factorielles, mais plutôt modifier un terme courant. Estimez la complexité de votre algorithme.

5. On dit d'un entier positif non nul n qu'il est un palindrome si on peut le lire indifféremment de gauche à droite, ou de droite à gauche. Par exemple, 3, 22, 121, 123321 et 4097904 sont des entiers palindromes.
   En vous limitant uniquement aux quatre opérateurs arithmétiques +, ×, / et %, écrivez le pseudocode de la fonction est_palindrome() qui retourne « vrai » si l'entier reçu en argument est un palindrome, « faux » sinon. Indication : Si on appelle C(y) le nombre de chiffres de y et R(y) le renversé d'un entier y, construisez itérativement deux entiers x et y tels que n=x*10^C(y)+R(y). Au début de l'itération, x=n et y=0, à la fin de l'itération x=0 et y=R(n). Éliminez tout de suite les multiples de 10 qui ne peuvent pas être des palindromes.